

Faculty of Computing
CS110: Fundamentals of Computer Programming
Class: BSAI/BSDS 2026A

Lab 05: Control Structures - Decision Making

CLO 1	Understand the syntax and semantics of different programming constructs
CLO 3	Build a program and associated documentation using appropriate IDE and supplementary tools

Time: 2:00 pm - 4:50 pm
Instructor: Sahar Arshad
Lab Engineer: <Add name>>

NAME: Abdur Rehman (Tariq)
CLASS/SECTION: BS AI-3-2026
CMS ID: 623486

Github Repo for all code/files:
<https://github.com/ARTariqDev/Nust-AI-2k26/tree/main/pf/LabTask05>



Introduction:

This lab moves from writing straight-line C++ programs to making programs choose what to do. Students will apply the selection structures introduced in the decision-making lecture: if, if-else, else-if, nested if, switch, and the ternary conditional operator. The three scenarios are different from the classroom examples so that the focus remains on understanding the shape of a decision rather than memorizing a familiar solution.

Learning Objectives:

After completing this lab, you will be able to:

- Write Boolean conditions using relational and logical operators and use a standalone if for an optional action.
- Use if-else for two-way decisions, an else-if ladder for multiple ranges, and nested if when one decision depends on another.
- Use switch for exact menu/command values and explain the purpose of break and default.
- Use the ternary operator for a short two-value choice without making the code difficult to read.
- Compile, test, trace, and document different branches of a C++ program using the IDE.

Tools/Software Requirement:

- The C++ IDE installed and configured in earlier labs (for example, Microsoft Visual Studio or the IDE specified by your instructor).
- A working C++ compiler integrated with the IDE.
- MS Word or equivalent for preparing the lab report.

Important: All three tasks are compulsory. Do not use loops in this lab; the focus is selection/decision structures.

Task 1 [CLO 1] - VR Lab Safety Gate: if and if-else (35 minutes):

A virtual-reality lab should not start a session unless the basic safety conditions are satisfied. Build a small decision program that checks the headset and play area before allowing the session to begin.

1. Declare and read: int headsetBattery (0-100), bool controllerConnected, and bool playAreaClear. Enter 1 for true and 0 for false when testing the Boolean values.
2. Use a standalone if statement to print "Battery warning: charge soon" whenever headsetBattery is below 20. This warning is an optional action and must be independent of the final start/block decision.
3. Use one if-else decision to print "VR session can start" only when headsetBattery $\geq$ 20, controllerConnected is true, and playAreaClear is true. Otherwise print "VR session blocked".
4. Inside the blocked path, use simple if statements to print specific reminders for any failed condition. Use ! at least once, for example to detect a disconnected controller.
5. Use meaningful labels in the output so the entered battery value and the final decision are clear.
6. **Required tests:** (75,1,1), (15,1,1), (60,0,1), and (60,1,0). Before running, predict whether the warning appears and whether the session starts or is blocked.
7. In 2-3 sentences, explain why the low-battery warning is written as a separate if instead of being the entire if-else decision.

Deliverable: C++ code, prediction table for the four tests, screenshots of at least two different branches, and the short explanation.



Task 2 [CLO 3] - Autonomous Drone Delivery Mission Advisor (55 minutes):

You are programming a beginner decision module for a small delivery drone. The drone receives mission details and must first describe the wind risk, then decide whether the mission can launch. This task uses both an else-if ladder and nested if decisions.

1. Read four values from the user: double packageWeight (kg), double distanceKm, int batteryLevel (0-100), and int windSpeed (km/h). Use meaningful identifiers exactly or equivalently.
2. Use an else-if ladder to classify the wind condition: 0-15 km/h -> "GREEN wind zone"; 16-30 km/h -> "YELLOW wind zone"; above 30 km/h -> "RED wind zone".
3. After printing the wind zone, make the mission decision using nested if statements. Follow this order so students can trace which tests are reached:
 - If windSpeed ≤ 30 , continue to the next decision; otherwise print "Mission postponed: wind too strong".
 - Inside the acceptable-wind branch, check batteryLevel ≥ 60 . If false, print "Recharge before mission".
 - Only when the battery check passes, check whether packageWeight ≤ 3.0 AND distanceKm ≤ 10.0 . If both are true, print "Launch approved"; otherwise print "Manual review: load or route exceeds mission limit".
4. Do not collapse the nested logic into one long condition. The purpose of this task is to show that later checks happen only after earlier checks succeed.

5. Required tests:

Test	Weight (kg)	Distance (km)	Battery	Wind
A	2.0	6	80	10
B	2.0	6	45	20
C	4.0	6	90	12
D	1.0	4	90	35

6. Before compiling, predict both outputs for each test: the wind-zone label and the final mission decision. Then run all four tests and compare your predictions with the program.

7. In your report, trace one test case in words by stating which condition is checked first, which nested condition is reached next, and why the final message is produced.

Deliverable: C++ code, predictions for Tests A-D, screenshots showing at least three different final decisions, and one short decision trace.



Task 3 [CLO 1] - Mars Rover Command Console: switch and ternary (55 minutes):

A Mars rover operator sends one command at a time. The command is an exact numeric option, so switch is the natural selection structure. The rover also displays a short power-status message using the ternary operator before the command is processed.

1. Read int command and int batteryLevel from the user. Assume batteryLevel is between 0 and 100.
2. Use one ternary expression to create or directly print the power status: batteryLevel >= 50 ? "POWER OK" : "LOW POWER". Do not use a nested ternary.
3. Use switch(command) with these exact options: 1 -> Capture panorama, 2 -> Collect rock sample, 3 -> Transmit data, 4 -> Return to base. Any other value must be handled by default as "Invalid rover command".
4. Place break after every valid case. Add a clear output label so the selected command and the power status can be read easily.

5. Required tests: (command=2, battery=68), (command=4, battery=25), and (command=9, battery=80). Predict the output before each run.

6. Debugging checkpoint: temporarily remove the break after case 1, run command=1, and record what extra output appears. Restore the break immediately afterward and explain the term fall-through in one sentence.

7. In 2-3 sentences, explain why switch is a better fit for the command number than an else-if ladder, and why ternary is suitable for the short power-status choice.

Deliverable: Completed C++ code, predictions for all required tests, screenshots of valid and invalid commands, the fall-through observation, and the switch/ternary explanation.

Deliverables:

Compile a single Word document and submit it on LMS. Organize the report under Task 1, Task 2, and Task 3. All three tasks are compulsory. For each task, include the requested C++ code, predictions/trace work, output or IDE screenshot(s). The submitted code must be the code demonstrated during the lab.



Lab Rubrics

Lab will be graded according to the following rubrics.

Lab Rubrics for Lab 5					
Sr. No.	Assessment	Unacceptable (0 Marks)	Does Not Meet Expectations (1/2 Marks)	Meets Expectations (3/4 Marks)	Exceeds Expectations (5 Marks)
1	Selection logic and control-structure semantics (CLO1, PLO1)	The student did not submit the required work, submitted work that cannot be evaluated, or used unfair means/plagiarized work.	The student is unable to form correct Boolean conditions or use if, if-else, else-if, nested if, switch, and ternary appropriately without substantial help. Programs contain major logic or structure errors.	The student demonstrates partial understanding of selection structures. Most scenarios are attempted and programs can run with limited guidance, although some conditions, branch ordering, break/default usage, or ternary choices are incorrect.	The student correctly selects and applies the required decision structures, writes accurate conditions, traces nested decisions, uses switch with break/default correctly, and explains program behavior across test cases.
2	IDE usage, testing, program construction, and documentation (CLO3, PLO5)	The student did not submit the required work, submitted work that cannot be evaluated, or used unfair means/plagiarized work.	The student has difficulty creating, building, and running the programs in the IDE and provides incomplete branch tests, predictions, screenshots, comments, or documentation.	The student can build and run the required programs and test several branches with limited help. Code, predictions, screenshots, comments, and explanations are mostly complete and organized.	The student independently builds, tests, traces, debugs, and documents all three programs; verifies different branches and edge cases; uses clear identifiers/comments; and presents complete, well-structured evidence.



Task 1

headsetBattery	contolerConnected	playareaClear	Predicted Output
75	1	1	"VR Session can start"
15	1	1	" Battery warning: charge soon VR session blocked See reasons below: Headset Battery is: 15% Please charge it to at least 20%"
60	0	1	"VR session blocked See reasons below: ERROR: please connect controller to begin session "
60	1	0	"VR session blocked See reasons below: ERROR: please clear the play area in order to begin the session"



Test 1 and Code screenshot:

```
task1.cpp X
LabTask05 > task1.cpp > main()
3  using namespace std;
4
5
6  int main () {
7      int headsetBattery; //0-100
8      bool controllerConnected, playAreaClear; //1 or 0
9
10     cout << "Enter headset Battery: ";
11     cin >> headsetBattery;
12     cout << "Is the Controller Connected? enter 1 or 0 for true and false respectively: ";
13     cin >> controllerConnected;
14     cout << "Is the play area clear? enter 1 or 0 for true and false respectively: ";
15     cin >> playAreaClear;
16
17     if (headsetBattery < 20) {
18         cout << "Battery warning: charge soon";
19     }
20
21     if (headsetBattery >= 20 && controllerConnected && playAreaClear) {
22         cout << "VR Session can Start";
23     } else {
24         cout << "VR session blocked";
25         cout << "See reasons below: ";
26         if (headsetBattery < 20) {
27             cout << "Headset battery is: " << headsetBattery << "%" << endl << "please charge it to at least 20% to begin";
28         }
29         if (!controllerConnected) {
30             cout << "ERROR: please connect controller to begin session" << endl;
31         }
32         if (!playAreaClear) {
33             cout << "ERROR: please clear the play area in order to begin the session";
34         }
35     }
36     return 0;
37 }
```

Problems Output Debug Console Terminal Ports Spell Checker 2

```
tariqmahmood@Tariqs-MacBook-Air LabTask05 % clang++ task1.cpp -o task1
tariqmahmood@Tariqs-MacBook-Air LabTask05 % ./task1
Enter headset Battery: 75
Is the Controller Connected? enter 1 or 0 for true and false respectively: 1
Is the play area clear? enter 1 or 0 for true and false respectively: 1
VR Session can Start
```

tariqmahmood@Tariqs-MacBook-Air LabTask05 %

Test 2 changed code to include more endls for formatting:



```
task1.cpp X
LabTask05 > task1.cpp > main()
3  using namespace std;
4
5
6  int main () {
7      int headsetBattery; //0-100
8      bool controllerConnected, playAreaClear; //1 or 0
9
10     cout << "Enter headset Battery: ";
11     cin >> headsetBattery;
12     cout << "Is the Controller Connected? enter 1 or 0 for true and false respectively: ";
13     cin >> controllerConnected;
14     cout << "Is the play area clear? enter 1 or 0 for true and false respectively: ";
15     cin >> playAreaClear;
16
17     if (headsetBattery < 20) {
18         cout << "Battery warning: charge soon" << endl;
19     }
20
21     if (headsetBattery >= 20 && controllerConnected && playAreaClear) {
22         cout << "VR Session can Start" << endl;
23     } else {
24         cout << "VR session blocked" << endl;
25         cout << "See reasons below: " << endl;
26         if (headsetBattery < 20) {
27             cout << "Headset battery is: " << headsetBattery << "%" << endl << "please charge it to";
28         }
29         if (!controllerConnected) {
30             cout << "ERROR: please connect controller to begin session" << endl;
31         }
32         if (!playAreaClear) {
33             cout << "ERROR: please clear the play area in order to begin the session";
34         }
35     }
36     return 0;
37 }
```

Problems Output Debug Console Terminal Ports Spell Checker 2

```
tariqmahmood@Tariqs-MacBook-Air LabTask05 % ./task1
Enter headset Battery: 15
Is the Controller Connected? enter 1 or 0 for true and false respectively: 1
Is the play area clear? enter 1 or 0 for true and false respectively: 1
Battery warning: charge soon
VR session blocked
See reasons below:
Headset battery is: 15%
please charge it to at least 20% to begin the session
tariqmahmood@Tariqs-MacBook-Air LabTask05 %
```

Screenshot



Test 3 (code remained same as test2):

```
Problems Output Debug Console Terminal Ports Spell Checker 2
tariqmahmood@Tariqs-MacBook-Air LabTask05 % ./task1
Enter headset Battery: 60
Is the Controller Connected? enter 1 or 0 for true and false respectively: 0
Is the play area clear? enter 1 or 0 for true and false respectively: 1
VR session blocked
See reasons below:
ERROR: please connect controller to begin session
tariqmahmood@Tariqs-MacBook-Air LabTask05 %
```

Test 4:

```
Problems Output Debug Console Terminal Ports Spell Checker 2
tariqmahmood@Tariqs-MacBook-Air LabTask05 % ./task1
Enter headset Battery: 60
Is the Controller Connected? enter 1 or 0 for true and false respectively: 1
Is the play area clear? enter 1 or 0 for true and false respectively: 0
VR session blocked
See reasons below:
ERROR: please clear the play area in order to begin the session%
tariqmahmood@Tariqs-MacBook-Air LabTask05 %
```

Low battery warning is written as a separate if statement since the user could have sufficient battery in the beginning but after starting the vr session, when the battery drops, the warning is displayed



Task 2

Code:

```
task1.cpp task2.cpp X
LabTask05 > task2.cpp > main()
5  int main () {
6
7      double packageWeight, distanceKm;
8      int batteryLevel, windSpeed;
9
10     cout << "Enter package weight: " << endl;
11     cin >> packageWeight;
12     cout << "Enter distance in Km: " << endl;
13     cin >> distanceKm;
14     cout << "Enter battery level: " << endl;
15     cin >> batteryLevel;
16     cout << "Enter windspeed: " << endl;
17     cin >> windSpeed;
18
19     if (windSpeed >= 30) {
20         cout << "RED wind zone" << endl;
21     }
22     else if (windSpeed >= 16) {
23         cout << "YELLOW wind zone" << endl;
24     }
25     else if (windSpeed <= 15 && windSpeed >= 0) { // adding compund condition to prevent negative values
26         cout << "GREEN wind zone" << endl;
27     }
28     else {
29         cout << "error, invalid input" << endl;
30     }
31     if (windSpeed <= 30) {
32         if (batteryLevel >= 60) {
33             if (packageWeight <= 3.0 && distanceKm <= 10.0) {
34                 cout << "Launch approved" << endl;
35             } else {
36                 cout << "Manual review: load or route exceeds mission limit" << endl;
37             }
38         } else {
39             cout << "Recharge before mission" << endl;
40         }
41     } else {
42         cout << "Mission postponed: wind too strong" << endl;
43     }
44     return 0;
45 }
```



Test Case A:

Prediction :

“GREEN wind zone

Launch approved”

Trace:

Since wind speed is in greenzone ($10 > 0$ and ≤ 15)

Afterwards: windspeed $\leq 30 \rightarrow$ pass $\rightarrow$ package weight = 6 and distance km = 2 so packageWeight ≤ 3.0 && distanceKm ≤ 10.0 passes $\rightarrow$ Launch approved is outputted

OUTPUT:

```
Problems Output Debug Console Terminal Ports Spell Checker 4
tariqmahmood@Tariqs-MacBook-Air LabTask05 % ./task1
Enter headset Battery: 60
Is the Controller Connected? enter 1 or 0 for true and false respectively: 1
Is the play area clear? enter 1 or 0 for true and false respectively: 0
VR session blocked
See reasons below:
ERROR: please clear the play area in order to begin the session%
tariqmahmood@Tariqs-MacBook-Air LabTask05 % clang++ task2.cpp -o ./task2
tariqmahmood@Tariqs-MacBook-Air LabTask05 % ./task2
Enter package weight:
2
Enter distance in Km:
6
Enter battery level:
80
Enter windspeed:
10
GREEN wind zone
Launch approved
❖ tariqmahmood@Tariqs-MacBook-Air LabTask05 %
```

Test Case B:

Prediction:

“

YELLOW wind zone

Recharge before mission

”



OUTPUT:

```
● tariqmahmood@Tariqs-MacBook-Air LabTask05 % ./task2
Enter package weight:
2
Enter distance in Km:
6
Enter battery level:
45
Enter windspeed:
20
YELLOW wind zone
Recharge before mission
❖❖ tariqmahmood@Tariqs-MacBook-Air LabTask05 % █
```

Test Case C:

Prediction:

"GREEN wind zone:

Manual review: load or route exceeds mission limit"

OUTPUT

```
● tariqmahmood@Tariqs-MacBook-Air LabTask05 % ./task2
Enter package weight:
4
Enter distance in Km:
6
Enter battery level:
90
Enter windspeed:
12
GREEN wind zone
Manual review: load or route exceeds mission limit
❖❖ tariqmahmood@Tariqs-MacBook-Air LabTask05 % █
```



Test Case D:

Prediction:

“

RED wind zone

Mission postponed: wind too strong

”

OUTPUT:

```
● tariqmahmood@Tariqs-MacBook-Air LabTask05 % ./task2
Enter package weight:
1
Enter distance in Km:
4
Enter battery level:
90
Enter windspeed:
35
RED wind zone
Mission postponed: wind too strong
❖ tariqmahmood@Tariqs-MacBook-Air LabTask05 % █
```



Task 3

CODE:

```
task1.cpp task2.cpp task3.cpp X
LabTask05 > task3.cpp > main()
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main () {
7      int command, batteryLevel;
8
9      cout << "Enter batterylevel (0-100): " << endl;
10     cin >> batteryLevel;
11
12     string output = batteryLevel >= 50 ? "POWER OK" : "LOW POWER";
13     cout << output << endl;
14     cout << "Enter Command (1-4): " << endl;
15     cout << "1 -> Capture panorama\n2 -> Collect rock sample\n3 -> Transmit data\n4 -> Return to base" << endl;
16     cin >> command;
17
18     switch (command) {
19         case 1:
20             cout << "Panorama Captured";
21             break;
22         case 2:
23             cout << "Rock Sample Collected";
24             break;
25         case 3:
26             cout << "Data Transmitted" ;
27             break;
28         case 4:
29             cout << "Returned to Base" ;
30             break;
31         default:
32             cout << "Invalid rover command";
33             break;
34     }
35
36     return 0;
37 }
```



Test Case 1:

Prediction:

“PoWER OK,
Rock Sample Collected”

OUTPUT:

```
● tariqmahmood@Tariqs-MacBook-Air LabTask05 % clang++ task3.cpp -o task3
● tariqmahmood@Tariqs-MacBook-Air LabTask05 % ./task3
Enter batterylevel (0-100):
68
POWER OK
Enter Command (1-4):
1 -> Capture panorama
2 -> Collect rock sample
3 -> Transmit data
4 -> Return to base
2
Rock Sample Collected%
❖ tariqmahmood@Tariqs-MacBook-Air LabTask05 %
```

Test Case 2:

Prediction:

“LOW POWER,
Returned to Base”

OUTPUT:

```
● tariqmahmood@Tariqs-MacBook-Air LabTask05 % ./task3
Enter batterylevel (0-100):
25
LOW POWER
Enter Command (1-4):
1 -> Capture panorama
2 -> Collect rock sample
3 -> Transmit data
4 -> Return to base
4
Returned to Base%
❖ tariqmahmood@Tariqs-MacBook-Air LabTask05 %
```



Test Case 3:

Prediction:

“POWER OK
Invalid Command”

OUTPUT:

```
● tariqmahmood@Tariqs-MacBook-Air LabTask05 % ./task3
Enter batterylevel (0-100):
80
POWER OK
Enter Command (1-4):
1 -> Capture panorama
2 -> Collect rock sample
3 -> Transmit data
4 -> Return to base
9
Invalid rover command%
❖❖ tariqmahmood@Tariqs-MacBook-Air LabTask05 %
```



Removing break for CASE 1:

The first output is with the break included, the second output is with the break removed. As you can see, by removing break, even though the condition passes, since there is no break, it falls through and prints the next case statement too.

```
Problems Output Debug Console Terminal Ports Spell Checker 5
tariqmahmood@Tariqs-MacBook-Air LabTask05 % clang++ task3.cpp -o task3
tariqmahmood@Tariqs-MacBook-Air LabTask05 % ./task3
Enter batterylevel (0-100):
55
POWER OK
Enter Command (1-4):
1 -> Capture panorama
2 -> Collect rock sample
3 -> Transmit data
4 -> Return to base
1
Panorama Captured%
tariqmahmood@Tariqs-MacBook-Air LabTask05 % clang++ task3.cpp -o task3
tariqmahmood@Tariqs-MacBook-Air LabTask05 % ./task3
Enter batterylevel (0-100):
55
POWER OK
Enter Command (1-4):
1 -> Capture panorama
2 -> Collect rock sample
3 -> Transmit data
4 -> Return to base
1
Panorama CapturedRock Sample Collected%
tariqmahmood@Tariqs-MacBook-Air LabTask05 %
```

Using a switch still serves as the better choice since the control structure centers around 1 variable only (command) so instead of multiple if-else ladders, we can make the code more concise by consolidation it to a switch.